



```
In [ ]: # --- Experiment: Implementing Transposed Convolution, Object Detection, and S
# Let's work on several tasks that will help you understand the details of con

# --- Task 1: Implement Transposed Convolution using numpy ---
import numpy as np
import matplotlib.pyplot as plt

def transposed_convolution(input_matrix, kernel, stride=1, padding=0):
    """
    Implement a basic transposed convolution (also called deconvolution) using

    Parameters:
    - input_matrix: 2D input feature map (numpy array), shape (H_in, W_in).
    - kernel: 2D convolution kernel (numpy array), shape (K_h, K_w).
    - stride: integer "stride" of the deconvolution.
    - padding: integer "padding" of the deconvolution.

    Returns:
    - 2D numpy array: Output of the transposed convolution.
    """
    input_matrix = np.asarray(input_matrix)
    kernel = np.asarray(kernel)

    if input_matrix.ndim != 2 or kernel.ndim != 2:
        raise ValueError("input_matrix and kernel must both be 2D arrays")
    if not isinstance(stride, int) or stride <= 0:
        raise ValueError("stride must be a positive integer")
    if padding < 0:
        raise ValueError("padding must be a non-negative integer")

    H_in, W_in = input_matrix.shape
    K_h, K_w = kernel.shape

    # Step 1: Insert zeros between and around input pixels
    # You could follow the following codes or crate similar logic by yourself
    # H_up = please determine the height of padded matrix
    # W_up = please determine the weight of padded matrix
    # padded = please initialize a zero matrix for padding (padding with zeros
    # how to combine with the original data?

    # Step 2: Initialize output size
    # H_out_final = please determine the height of final output matrix
    # W_out_final = please determine the height of final output matrix
    # output_final = please initialize a zero matrix

    # Step 3: Perform transposed convolution
    # for i in range():
    #     for j in range():
    #         Here, implement the core logic for applying kernel to each input
    # return output_final

# Test the function with a simple input matrix and kernel
input_matrix = np.array([[1, 2], [3, 4]], dtype=np.float64)
```

```

kernel = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=np.float64)
output = transposed_convolution(input_matrix, kernel, stride=2, padding=1)
print(f"{'#' * 30}\n Test Sample 1: \n", output)

# more test samples
input_matrix = np.array([[2, 5, 6], [1, 3, 4]], dtype=np.float64)
kernel = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=np.float64)
output = transposed_convolution(input_matrix, kernel, stride=2, padding=1)
print(f"{'#' * 30}\n Test Sample 2: \n", output)
output = transposed_convolution(input_matrix, kernel, stride=3, padding=1)
print(f"{'#' * 30}\n Test Sample 3: \n", output)
output = transposed_convolution(input_matrix, kernel, stride=1, padding=0)
print(f"{'#' * 30}\n Test Sample 4: \n", output)
output = transposed_convolution(input_matrix, kernel, stride=2, padding=0)
print(f"{'#' * 30}\n Test Sample 5: \n", output)

# Plot input and output
plt.subplot(1, 2, 1)
plt.title("Input Matrix")
plt.imshow(input_matrix, cmap='gray')

plt.subplot(1, 2, 2)
plt.title("Transposed Convolution Output")
plt.imshow(output, cmap='gray')
plt.show()

```

```

In [ ]: # --- Task 2: Use Pretrained DETR and U-Net for Object Detection and Semantic
import torch
from torchvision import transforms
from transformers import DetrImageProcessor, DetrForObjectDetection

# Load a pretrained DETR model and use it for object detection
def detect_objects_with_detr(image_path):
    image = Image.open(image_path).convert("RGB")
    model = torch.hub.load('facebookresearch/detr', 'detr_resnet50', pretrained=True)
    model.eval();

    # Step 1: define the preprocessing
    # transform = T.Compose([])

    # Bonus 1: use lists and hooks to store the intermediate outputs

    # Step 2: apply preprocessing and model inference
    # TODO: finish the code snippet here
    # input is "image", apply preprocessing and feed to model, output will be

    # Step 3: post-processing, only keep high confidence predictions, rescale
    # TODO: obtain probas (probability) for each object, filter low probability

    # Step 4: Visualize the results
    # TODO: Implement code to visualize the detected objects and their labels

    # Bonus 2: fetch and visualize intermediate feature and attention map

```

```
pass
```

```
# Call detect_objects_with_detr with an example image  
detect_objects_with_detr("img110.png")
```

```
In [ ]: # --- Task 3: Use Pretrained U-Net for Semantic Segmentation ---  
import torchvision.models.segmentation as seg_models  
  
def segment_image_with_unet(image_path):  
    # TODO Step 1: Load the pretrained U-Net model  
    # model =  
  
    # TODO Step 2: define the preprocessing  
    # transform =  
    # image = None # Load the image  
  
    # TODO Step 3: Run the model on the image  
    # output =  
    # output_predictions =  
  
    # TODO Step 4: Visualize the segmentation result  
    pass  
  
# TODO: Call segment_image_with_unet with an example image  
# segment_image_with_unet("path_to_image.jpg")
```